Expectation–Maximization for RSS Noise Calibration in a Synthetic Radio Tomographic Imaging Study

Camilo Vásquez
School of Mathematical and Computational Sciences
Yachay Tech University
Urcuquí, Ecuador

***Abstract*—This study examines Expectation–Maximization (EM) as an algorithm for estimating a two-component, zero-mean Gaussian mixture used to describe received-signal-strength (RSS) noise. A synthetic radio tomographic imaging (RTI) model provides the application context. The study separates parameter recovery from localization: EM is evaluated across five calibration sample sizes and 20 initializations, whereas classical Tikhonov RTI, a scalar Fisher-information extension, a weighted nearest-neighbor baseline, and a constant room-center control are evaluated on 500 paired generated cases. Parameter recovery generally improves with sample size under the correctly specified generating model. Mean localization errors are 3.244, 3.062, 2.618, and 2.407 m, respectively. All signal-dependent methods therefore lose to the constant control in mean error. Algebra shows that the Fisher extension merely changes the effective Tikhonov regularization from α to α/γ; it does not provide link-specific robustness. The results support a reproducible implementation of EM but do not establish useful localization, superiority over nearest neighbors, or real Wi-Fi sensing performance. The report provides the EM derivation, a numerical iteration, pseudocode, complexity analysis, traceable results, and a plan for independent development and test evaluation. The EM algorithm is implemented in C++17; Python supplies data generation, RTI, validation, and plotting utilities.**

***Index Terms*—expectation–maximization, Gaussian mixture, RSS, radio tomographic imaging, parameter estimation, synthetic evaluation**

I. Introduction

Radio tomographic imaging estimates changes in spatial attenuation from signal-strength changes across a network of radio links. An object may alter several links without carrying a transmitter. Wilson and Patwari formulated this setting as an ill-posed linear inverse problem and demonstrated image reconstruction using a deployed network of 28 radio nodes [1]. That application motivates this project, but its principal subject is the Expectation–Maximization algorithm, rather than the engineering of a deployable sensing system.

EM estimates parameters in models with unobserved variables by alternating a conditional-expectation calculation and a parameter update [2]. In a mixture model, the missing variable is the component that generated each observation. The observed residual is available; its component label is not. This distinction matters because estimating latent labels is different from imputing missing RSS measurements. No missing measurements are imputed in the present implementation.

The project studies a particularly small model: a mixture of two normal distributions whose means are fixed at zero. One component has a narrower spread, the other a wider spread. The parameters to estimate are one mixing probability and two standard deviations. Both EM steps admit explicit formulas and one iteration can be checked numerically. The restricted model also makes initialization, convergence criteria, numerical stability, and the distinction between density estimation and application validation directly inspectable.

The application begins with a geometric matrix linking 100 image pixels to 66 radio links. Synthetic observations are generated by adding mixture noise to the attenuation predicted by that matrix. EM is fitted to a separate synthetic calibration sample. The estimated density then produces a scalar Fisher information, γ. An exploratory RTI operator inserts that scalar into regularized normal equations. This construction is explicitly distinguished from the source article's classical Tikhonov estimator and from an exact mixture-likelihood estimator.

The initial comparison contained three signal-dependent methods. Adding a constant room-center control changes the interpretation: the best signal-dependent result still has a larger mean distance error than a predictor that uses no signals. An audit must retain that result rather than selecting only the comparison in which EM appears favorable. Improving one RTI configuration is insufficient evidence that the resulting localization is useful.

This report addresses three research questions. RQ1 asks how calibration sample size affects EM recovery of known mixture parameters. RQ2 asks whether different initializations on one fixed sample give different fitted solutions. RQ3 asks what the generated localization comparison actually establishes after including a spatial control and identifying the algebraic effect of γ. These questions can be answered using the available data without claiming electromagnetic simulation, real deployment performance, or an algorithmic innovation.

The implementation provides a C++17 EM core, repeated calibration diagnostics, a paired synthetic localization evaluation, and editable scientific documentation. Its practical boundary is precise: no hardware, human measurements, walls, rubble, propagation solver, channel-state information (CSI), angle of arrival, or time of arrival are involved. C++ executes the calibration algorithm; auxiliary Python utilities prepare

observations and evaluate the surrounding RTI application.

II. Background and Scope of the Evidence

A. EM and incomplete data

Dempster, Laird, and Rubin introduced EM as a general framework for maximum-likelihood estimation from incomplete data [2]. The framework covers several forms of incompleteness. A data table may contain missing entries, but incompleteness may instead arise because the generative model includes a hidden class or state. The latter interpretation applies here. Every calibration residual is observed, and EM infers its probabilistic membership in the two noise components.

Exact EM updates do not decrease the observed-data likelihood. This useful property is weaker than a guarantee of finding the global maximum. The likelihood surface may have stationary points, flat regions, equivalent labelings, or boundary solutions. Wu's analysis distinguishes convergence of likelihood values from convergence of the parameter sequence and establishes results under explicit conditions [3]. Consequently, a program must report its stopping criterion and iteration limit rather than interpreting any returned parameter vector as a certified optimum.

Fixing both means to zero removes two location parameters. It also changes the geometry of the optimization problem relative to a general Gaussian mixture. Conclusions about singularities, component collapse, or identifiability must therefore be stated for this particular model. The numerical safeguards and the ordering convention used in this project are described in Section VI.

B. RTI and the source experiment

Wilson and Patwari represent link-wise RSS changes by a linear model, use ellipse-based pixel weights, investigate a two-component noise mixture, derive image-error bounds, and reconstruct images with Tikhonov regularization [1]. These are separate contributions within the article. The Fisher scalar appears in the bound on estimation error. It does not appear in their later classical Tikhonov reconstruction formula.

The source experiment employed TelosB nodes using IEEE 802.15.4 in the 2.4-GHz band [1]. Sharing a frequency band does not make this network Wi-Fi: IEEE 802.11 and IEEE 802.15.4 are different radio standards. In this project, "RSS-like" denotes generated signal-strength residuals. It does not imply that the script simulates either standard's physical layer or acquires data from a router or a phone.

The source's noise study pools link measurements after removing each link's mean. The resulting mixture estimates were $p_1 = 0.548$, $\sigma_1 = 0.971$ dB, and $\sigma_2 = 3.003$ dB. The current generator adopts those published values as controlled parameters. It does not independently establish that these values describe another room, device, Wi-Fi network, or deployment. Copying parameters defines a simulation condition; it is not a new empirical measurement.

C. Fingerprinting and the comparison baseline

Fingerprint localization compares a query signal pattern with reference patterns acquired at known positions. RADAR is an early radio-frequency location system using measured signal strengths and a position database [4]. Its role here is background for fingerprinting, not evidence that the present synthetic nearest-neighbor baseline reproduces that physical system.

The implemented baseline is more precisely weighted k-nearest neighbors: it selects the three closest training fingerprints and forms an inverse-distance weighted average of their coordinates. Both its reference fingerprints and test observations come from the same synthetic forward model. Noise draws are independent, but the spatial grid and environment model are shared. This comparison evaluates matching under a known generated environment; it does not test generalization to an unseen building.

D. What the study can establish

The data allow direct checks of implementation arithmetic, recovery under a correctly specified generating distribution, sensitivity to sample size and initial values, and the behavior of specific reconstruction settings. They do not identify how much of the behavior would persist under channel-model mismatch. In particular, a two-variance mixture is not a complete model of multipath propagation, and generating independent wide-component samples does not demonstrate resistance to non-line-of-sight (NLoS) obstruction.

No result from recent CSI-based, multi-access-point localization work is transferred to the present experiment. Such methods can involve richer observations and different estimators. Their reported performance cannot justify a numerical accuracy claim for a fixed RSS ellipse model. The evidence chain in this report remains limited to the source RTI formulation, the standard EM framework, and the local synthetic output files.

III. Problem Definition and Data Flow

A. Observed and unobserved quantities

Let $r_i \in \mathbb{R}$ denote calibration residual i, for $i = 1, \ldots, N$. The residual is expressed in dB and assumed to be centered at zero. Let $z_i \in \{1, 2\}$ identify the latent mixture component. The unknown parameter vector is

$$\theta = (p, \sigma_1, \sigma_2), \quad 0 < p < 1, \quad 0 < \sigma_1 \leq \sigma_2. \tag{1}$$

The ordering distinguishes the narrower component from the wider component; without it, swapping labels represents the same density.

The complete calibration data would contain both r_i and z_i. Only r_i is supplied to EM. Although the generator internally draws component labels, those labels are not used by the fitting function. The generating parameters are used later to score recovery; they are not supplied as true values to the estimator. Separating these roles prevents a trivial evaluation in which an estimator is handed the quantity it is meant to infer.

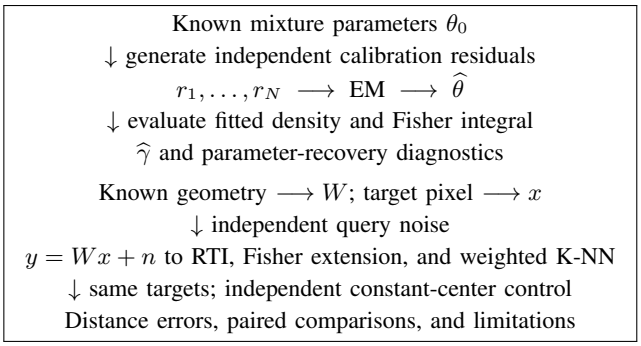


Fig. 1. Two connected but distinct experiments. Calibration evaluates EM parameter estimation. Localization evaluates fixed predictors on generated queries. The center control uses no query signal. All arrows denote computation, not physical radio acquisition.

The localization experiment has a different unknown: an attenuation image $x \in \mathbb{R}^V$. Its observed input is a link vector $y \in \mathbb{R}^M$. Geometry determines a matrix $W \in \mathbb{R}^{M \times V}$. EM does not directly estimate the target position. Instead, its fitted noise parameters enter one exploratory reconstruction setting through γ. The classical RTI baseline and the nearest-neighbor baseline operate without fitting a latent target class by EM.

B. Independence and conditioning

Calibration residuals, fingerprint-training noise, and query noise are separate random draws. The same query vector is passed to every signal-dependent method in a localization case. This pairing controls which noise realization each method sees. It does not make the methods identical in available information: the fingerprint method has a labeled position database, whereas RTI uses the geometric matrix and a smoothing prior.

The main localization evaluation conditions on one geometry and one fitted calibration density. Bootstrap intervals therefore describe variation across generated query cases under that fixed configuration. They do not include uncertainty from selecting another room, recollecting calibration, changing devices, or refitting the density. Keeping the conditioning explicit is essential when interpreting a narrow interval.

C. Model checks before fitting real data

Applying this pipeline to real measurements would require link-wise centering, acquisition checks, and an assessment of temporal dependence before fitting the mixture. A residual mean far from zero contradicts the fixed-mean assumption. Several links with different offset distributions may require link-specific treatment. Repeated fading intervals can make consecutive residuals dependent even when their one-dimensional histogram resembles a mixture.

The present synthetic generator deliberately avoids those complications. It supplies independent residuals from the fitting model, which is appropriate for a first implementation check. Correct performance in this setting establishes that the estimator can recover a distribution it was designed to fit. The next scientific question would be how recovery changes under departures from that model, rather than whether successful recovery already validates the sensing system.

IV. Derivation of the EM Algorithm

A. Observed-data likelihood

Define the zero-mean normal density by

$$\phi(r; 0, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{r^2}{2\sigma^2}\right). \tag{2}$$

The mixture density is

$$f(r; \theta) = p\phi(r; 0, \sigma_1^2) + (1-p)\phi(r; 0, \sigma_2^2). \tag{3}$$

Assuming independent residuals, the observed-data log likelihood is

$$\ell(\theta) = \sum_{i=1}^{N} \log f(r_i; \theta). \tag{4}$$

The difficulty is the logarithm of a sum. A component's contribution depends on all three parameters, so treating component membership as known would give incorrect parameter updates. EM resolves this by estimating probabilistic membership from the current parameters and optimizing an expected complete-data expression.

B. Complete-data likelihood

Let $s_i = 1$ when $z_i = 1$ and $s_i = 0$ otherwise. If these indicators were observed, the log likelihood, after discarding constants independent of θ, would be

$$\begin{aligned} \ell_c(\theta) = \sum_i \Bigg[& s_i \log p + (1 - s_i) \log(1-p) \\ & - s_i \log \sigma_1 - (1 - s_i) \log \sigma_2 \\ & - \frac{s_i r_i^2}{2\sigma_1^2} - \frac{(1-s_i) r_i^2}{2\sigma_2^2} \Bigg]. \end{aligned} \tag{5}$$

Each component now has a weighted sum of squared residuals. Because its mean is fixed at zero, the sufficient second moment is r_i^2, not the squared distance from a newly estimated mean. Using a general Gaussian-mixture formula that updates component means would implement a different model.

At iteration t, EM forms

$$Q(\theta \mid \theta^{(t)}) = \mathbb{E}_{z|r,\theta^{(t)}}[\ell_c(\theta)]. \tag{6}$$

The E-step evaluates this conditional expectation through the component responsibilities. The M-step maximizes it over the new parameters while holding the responsibilities fixed. "Expectation" therefore refers to the unobserved indicators conditional on data; it does not mean replacing every residual by its unconditional average.

C. E-step: responsibilities

Bayes' rule gives the probability that residual i arose from the narrower component:

$$\tau_i^{(t)} = \frac{p^{(t)} \phi(r_i; 0, (\sigma_1^{(t)})^2)}{p^{(t)} \phi(r_i; 0, (\sigma_1^{(t)})^2) + (1 - p^{(t)}) \phi(r_i; 0, (\sigma_2^{(t)})^2)}. \tag{7}$$

The wider component receives responsibility $1 - \tau_i^{(t)}$. These numbers are soft assignments in $[0, 1]$. A residual near zero is generally more compatible with the narrower density, while

a sufficiently large residual tends to favor the wider one. The mixing probabilities also influence this comparison, so a responsibility cannot be inferred from residual magnitude alone.

Two residuals with the same absolute value have identical responsibilities in this model. Their signs do not distinguish components because both densities are symmetric about zero. This property offers a simple implementation check and makes the small worked example below easy to inspect.

D. M-step: mixing probability

Write $N_1 = \sum_i \tau_i$ and $N_2 = N - N_1$. The terms in Q involving p are $N_1 \log p + N_2 \log(1-p)$. Differentiating gives

$$\frac{\partial Q}{\partial p} = \frac{N_1}{p} - \frac{N_2}{1-p} = 0. \tag{8}$$

Solving yields

$$p^{(t+1)} = \frac{N_1}{N}. \tag{9}$$

The new probability is the average fractional membership in component 1. It is not a count of residuals assigned by a hard threshold. This distinction avoids information loss when an observation has substantial support under both components.

E. M-step: variances

For $v_1 = \sigma_1^2$, the relevant expected terms are

$$Q_1(v_1) = -\frac{N_1}{2} \log v_1 - \frac{1}{2v_1} \sum_i \tau_i r_i^2. \tag{10}$$

Its derivative is $-N_1/(2v_1) + \sum_i \tau_i r_i^2/(2v_1^2)$. Setting it to zero, and repeating for component 2, produces

$$(\sigma_1^{(t+1)})^2 = \frac{\sum_i \tau_i^{(t)} r_i^2}{N_1}, \tag{11}$$

$$(\sigma_2^{(t+1)})^2 = \frac{\sum_i (1-\tau_i^{(t)}) r_i^2}{N_2}. \tag{12}$$

The implementation stores standard deviations, so it takes square roots after updating the weighted second moments. Reporting a variance as though it were a standard deviation would change both units and the density shape. Table I summarizes the relevant quantities.

F. Why the iteration increases likelihood

The conditional distribution of z defined by the old parameters provides a lower-bound construction for the observed log likelihood. The E-step makes that bound tight at the current parameter vector; the M-step increases its expected complete-data term. For exact maximization on a fixed admissible parameter domain, the resulting observed likelihood does not decrease [2].

This explanation identifies what must be checked in code. The responsibilities used for every M-step update must all come from the same old parameter vector. Updating one parameter and immediately using it to recompute another responsibility inside the same iteration would no longer be the stated EM update. Likewise, comparing likelihood values from different datasets says nothing about monotonicity on a particular fit.

The implementation's probability clipping, minimum standard deviations, and finite arithmetic define a restricted numerical procedure. A likelihood trace is therefore useful for auditing the actual computation rather than relying solely on the abstract theorem. A small parameter change can occur in a flat region, and reaching the iteration limit is different from satisfying the stopping condition. Neither event alone certifies the best possible solution.

TABLE I
PRINCIPAL QUANTITIES AND THEIR ROLES

Symbol	Meaning and units
$N, I, I_{\max}$	Residual count, executed iterations, and iteration limit
p, τ_i	Mixing probability and conditional responsibility
r_i, σ_k	Residual and standard deviation; dB
σ_k^2	Component variance; dB^2
M, V	Number of radio links and image pixels
x, y, W	Attenuation image, RSS-change vector, geometric matrix
α	Smoothness weight in the implemented RTI operator
γ	Location Fisher information of the residual density; dB^{-2}

V. Worked Iteration and Algorithm Structure

A. A numerical example

Consider the four residuals $(-3, -1, 1, 3)$ dB and initial values $p = .5$, $\sigma_1 = 1$, $\sigma_2 = 3$ dB. This numerical illustration concerns one iteration and is separate from the 500 localization cases. It can be reproduced directly with the density and update equations.

At absolute residual 1, the two unweighted densities are approximately .241971 and .125794 dB^{-1}. With equal mixing probabilities, their normalized ratio gives $\tau(1) = .657949$. At absolute residual 3, the densities are approximately .004432 and .080657 dB^{-1}, giving $\tau(3) = .052085$. Symmetry supplies the same values at negative residuals.

The effective narrow mass is $N_1 = 2(.657949 + .052085) = 1.420068$. Therefore the updated weight is .355017. The variance updates are

$$v_1^{\mathrm{new}} = \frac{2(.657949)(1) + 2(.052085)(9)}{1.420068} \approx 1.586845, \tag{13}$$

$$v_2^{\mathrm{new}} = \frac{2(.342051)(1) + 2(.947915)(9)}{2.579932} \approx 6.878698. \tag{14}$$

Taking square roots yields $\sigma_1^{\mathrm{new}} = 1.259700$ and $\sigma_2^{\mathrm{new}} = 2.622727$ dB. The E-step favored the wide component strongly for the two large residuals, and the M-step converted those fractional memberships into a lower narrow-component probability and updated spreads.

EM does not eliminate the large residuals in this example. They remain observed values and contribute to both parameter estimates according to their responsibilities. No geometry or

TABLE II
PSEUDOCODE FOR THE RESTRICTED EM FIT

Input: finite residuals $r[1..N]$; limit $I_{\max}$; tolerance ε; spread floor $s_{\min}$.
Initialize $\theta \leftarrow (p, \sigma_1, \sigma_2)$; set $\delta = 10^{-8}$.
for $t \leftarrow 1$ **to** $I_{\max}$:
  $\theta_{\text{old}} \leftarrow \theta$
  E-step: **for** $i \leftarrow 1$ **to** N:
    $a \leftarrow p\phi(r_i; 0, \sigma_1^2)$
    $b \leftarrow (1-p)\phi(r_i; 0, \sigma_2^2)$
    $\tau_i \leftarrow a/(a+b)$
  M-step:
  $N_1 \leftarrow \max(\sum_i \tau_i, \delta)$; $N_2 \leftarrow \max(N - N_1, \delta)$
  $p \leftarrow \text{clip}(N_1/N, 10^{-4}, 1 - 10^{-4})$
  $\sigma_1 \leftarrow \sqrt{\sum_i \tau_i r_i^2 / N_1}$
  $\sigma_2 \leftarrow \sqrt{\sum_i (1-\tau_i) r_i^2 / N_2}$
  Apply $\sigma_k \geq s_{\min}$ and order components.
  If spreads swap, also set $p \leftarrow 1 - p$.
  $\theta \leftarrow (p, \sigma_1, \sigma_2)$
  if $\max_k |\theta_k - \theta_{\text{old},k}| / \max(|\theta_{\text{old},k}|, \delta) < \varepsilon$:
    return θ, t, true
return $\theta, I_{\max}, \text{false}$

position variable appears in the calibration updates; target localization belongs to the subsequent RTI calculation.

B. Pseudocode

Algorithm II states the mathematical iteration using normal densities ϕ to expose the E-step and M-step. The executed C++ routine evaluates log densities and normalizes them stably, as described in Section VI. The density notation below is an explanatory expression, not a replacement for that stable implementation. Component masses are protected before division, and the updated parameters are bounded and ordered before the stopping test.

C. Time and memory complexity

Let N be the number of valid scalar residuals, I the number of iterations actually executed, and $I_{\max}$ the configured limit. The analysis assumes two fixed components and fixed-precision arithmetic. A representative basic operation is evaluating one sample's density, or log density, for one component. The E-step performs two such evaluations per sample and iteration, so its representative count is

$$C_E(N, I) = \sum_{t=1}^{I} \sum_{i=1}^{N} \sum_{k=1}^{2} 1 = 2IN. \tag{15}$$

This counts sample–component evaluations. It is not an exact count of all arithmetic operations: normalization, exponentials, divisions, and the M-step also contribute work.

Input validation/filtering and dispersion-based initialization take $\Theta(N)$ time. Within an iteration, the E-step traverses the residuals in $\Theta(N)$ time; the M-step computes effective counts and weighted moments in $\Theta(N)$ time; and the three-parameter stopping check, bounds, and label ordering take $\Theta(1)$ time. Therefore, for $I \geq 1$,

$$\begin{aligned} T(N, I) &= \Theta(N) + I[\Theta(N) + \Theta(N) + \Theta(1)] \\ &= \Theta(IN), \qquad T(N, I) \in O(I_{\max} N). \end{aligned} \tag{16}$$

The E-step and M-step are successive passes, so their costs add. They are not two nested loops over N and do not produce an N^2 cost. The loop over two components in Eq. (15) has fixed size.

One iteration is linear in N; the full fit is not simply "always $O(N)$." Its executed iteration count depends on the data, initialization, and stopping tolerance ε. For fixed I, time grows linearly with N. Repeated fits add costs: for S starts, $\Theta(N \sum_{s=1}^{S} I_s)$ accounts for potentially different iteration counts. The saved configuration uses $I_{\max} = 500$ and $\varepsilon = 10^{-7}$.

The C++ routine stores a responsibility vector `tau` of length N, giving $\Theta(N)$ auxiliary memory beyond its input vector. Component densities, weighted sums, three parameters, iteration index, and returned result require $\Theta(1)$ space. The input itself occupies $\Theta(N)$ space. These bounds apply to one EM fit and exclude RTI matrices, fingerprints, plots, process startup, and file handling. Observed runtimes also depend on compiler settings and interface overhead; they are not a proof of the asymptotic operation count.

VI. NUMERICAL IMPLEMENTATION

A. Stable normalization

Directly evaluating exponentials can underflow for a residual that is large relative to a component spread. The implementation first computes

$$a_{ik} = \log p_k - \tfrac{1}{2} \log(2\pi\sigma_k^2) - \frac{r_i^2}{2\sigma_k^2}. \tag{17}$$

With $m_i = \max(a_{i1}, a_{i2})$, the responsibility is evaluated as

$$\tau_i = \frac{\exp(a_{i1} - m_i)}{\exp(a_{i1} - m_i) + \exp(a_{i2} - m_i)}. \tag{18}$$

Subtracting the same maximum cancels algebraically in the ratio while keeping the largest exponent at zero. The observed log density can similarly be computed as $m_i + \log\{\exp(a_{i1} - m_i) + \exp(a_{i2} - m_i)\}$. This implements the same statistical model with more stable arithmetic.

B. Initialization, labels, and stopping

The default mixing weight begins at .7. The initial narrow spread is .55 times the empirical sample spread, bounded below by .15 dB. The wider spread begins at 1.8 times that spread estimate and is kept larger than the narrow initial value. The diagnostic additionally supplies randomized initial values to the same fitting routine. These are initialization rules, not estimated physical constants.

At each iteration the code computes the maximum relative parameter change, using a small denominator safeguard when an old value is near zero. The fit is declared converged when this value is below 10^{-7}; otherwise it stops at the configured

iteration limit. The report distinguishes a returned estimate from the convergence flag. A nonconverged fit may still have reasonable parameters, but it should not be silently counted as a successful stopping event.

After an M-step the components are ordered by spread. If the labels swap, the mixing weight is replaced by $1 - p$ to preserve the represented density. This avoids treating the same solution under opposite labels as two different recovered parameter vectors. The ordering does not force well-separated components: two estimated spreads can still become close, in which case the mixture has little practical distinction between its components.

C. Floors and the zero-mean restriction

The program restricts standard deviations to at least .15 dB and keeps mixing probabilities away from exact zero and one. These choices protect divisions and define an admissible numerical domain. They can affect a fit near the boundary, so their values should be recorded rather than treated as an invisible implementation detail.

In a Gaussian mixture with freely estimated means, the likelihood can become unbounded when one component collapses onto an observation. That generic explanation must be qualified for the model used here: both means are fixed at zero. For a fixed nonzero residual r, maximizing $\phi(r; 0, \sigma^2)$ over $\sigma > 0$ gives

$$\sup_{\sigma>0} \phi(r; 0, \sigma^2) = \frac{\exp(-1/2)}{\sqrt{2\pi}|r|} < \infty. \tag{19}$$

Every mixture density at that nonzero residual is bounded by the same maximum. If all sample values are nonzero, their likelihood is therefore bounded, even without movable means. An exact zero changes the situation: a positive-weight component centered at zero can have density diverging as its spread approaches zero while the other component explains the remaining observations.

Thus the floor is retained as a numerical and modeling restriction, including protection for exact zeros that may occur after RSS quantization or centering. The project does not claim that a free-mean singularity occurs at an arbitrary nonzero observation in its own fixed-mean model. Sensitivity to the floor remains a future experiment; it is not measured by the current sample-size comparison.

D. Separation between core and experiment driver

The standalone `em.cpp` source contains the assigned E-step and M-step using explicit loops and standard C++17 facilities. It is compiled with `g++`; an auxiliary `em_cpp.py` adapter supplies residuals and reads the returned parameters. The main calibration fit, sample-size diagnostics, and initialization diagnostics use that compiled backend. Geometry, random-data generation, RTI reconstruction, bootstrap summaries, and plotting remain Python utilities outside the assigned EM core.

The C++ core in Appendix A is taken from the editable source packaged with this report. Command-line parsing and the numerical demonstration surround that function in the full source; they are not additional EM steps. The Python implementation in `validation/em_reference.py` is retained for numerical cross-checks. The selected experimental backend remains C++, and the estimation loop is separated from the surrounding experimental driver.

VII. Synthetic RTI and Comparison Methods

A. Geometry and forward model

The room is a 6.4-m square with 12 evenly spaced perimeter nodes. Every node pair supplies an available undirected link, giving $M = 12(11)/2 = 66$. The area is divided into a 10×10 grid, so $V = 100$ and each pixel is .64 m wide. The count of links is smaller than the count of unknown pixels; the inverse problem therefore requires additional structure.

Let d_l be the distance between a link's nodes, and $d_{lj}^{(1)}$, $d_{lj}^{(2)}$ the distances from pixel center j to the two endpoints. The implemented ellipse weights follow the source-style rule

$$W_{lj} = \begin{cases} d_l^{-1/2}, & d_{lj}^{(1)} + d_{lj}^{(2)} < d_l + \lambda, \\ 0, & \text{otherwise}, \end{cases} \tag{20}$$

with $\lambda = .10$ m. This matrix is a statistical sensitivity model. It is not a numerical solution to Maxwell's equations and does not compute reflection, material penetration, diffraction, or packet decoding.

The true image has one nonzero pixel with attenuation 4 dB. A query is generated by

$$y = Wx + n, \tag{21}$$

where each element of n is an independent mixture draw. This single-pixel target simplifies attribution of localization error but does not represent a complete human attenuation pattern. The source article's physical image experiment and evaluation used a different target representation and acquisition process [1].

B. Classical Tikhonov baseline

Let D_x and D_y denote adjacent-grid difference operators and define $L = D_x^{\mathsf{T}} D_x + D_y^{\mathsf{T}} D_y$. The baseline reconstructs

$$\widehat{x}_T = (W^{\mathsf{T}} W + \alpha L + \epsilon I)^{-1} W^{\mathsf{T}} y, \tag{22}$$

with $\alpha = .10$ and a small numerical stabilizer $\epsilon = 10^{-8}$. The image is clipped below zero, and the coordinate of its maximum is the predicted target location. Clipping is an explicit postprocessing choice; it is not the exact solution of a separately optimized nonnegative constrained problem.

For the default grid the script precomputes the dense linear operator. Inference then multiplies that operator by each query vector. The setup includes matrix construction and factorization or inversion, whereas inference applies an already prepared operator. These phases must not be mixed when discussing computational cost. At larger grids, the driver can use an iterative solver, which has a different setup and per-query profile.

C. Fisher scalar and exploratory extension

For a location-noise density f, the scalar Fisher information is

$$\gamma = \int_{-\infty}^{\infty} \frac{[f'(u)]^2}{f(u)}\, du. \tag{23}$$

Under the noise assumptions in the source, the data-information matrix is $J_D = \gamma W^{\mathsf{T}} W$. The article combines this term with prior information to form an image-error bound [1]. A bound on possible estimation error is not itself an image estimator.

The project's exploratory operator is

$$\widehat{x}_F = (\gamma W^{\mathsf{T}} W + \alpha L + \epsilon I)^{-1} \gamma W^{\mathsf{T}} y. \tag{24}$$

For positive γ, dividing the normal equations by it gives the exact identity

$$\widehat{x}_F = \left(W^{\mathsf{T}} W + \frac{\alpha}{\gamma} L + \frac{\epsilon}{\gamma} I \right)^{-1} W^{\mathsf{T}} y. \tag{25}$$

Consequently, this operator belongs to the same scalar Tikhonov family. Its main calibration produces $\gamma = .4182318686$, giving $\alpha/\gamma = .2391018$ rather than .10. The comparison changes the amount of smoothing. All links share the same scalar multiplier, so this construction does not identify or downweight a particular contaminated link.

The equivalence survives the subsequent common clipping and argmax operations because the reconstructed vectors are equal under the corresponding parameters. A future tuned comparison must include the baseline at $\alpha = .2391018$, with the corresponding stabilizer, as an algebraic control. Calling the present operator an exact mixture maximum-likelihood or robust NLoS estimator would misdescribe what the code computes.

D. Weighted nearest neighbors and center control

For each grid cell the driver creates three noisy reference fingerprints using the same W and attenuation model. The query's nearest three vectors are selected by Euclidean signal distance. Their coordinates are averaged with inverse-distance weights, using a small distance safeguard. The label KNN in result files is retained for consistency; the report identifies its weighted prediction rule explicitly.

The constant control always returns the geometric room center $(3.2, 3.2)$ m. It has no training data, no radio observations, and no calibration parameters. For uniformly sampled positions in a bounded square, it provides a meaningful check against location predictions that are effectively dispersed or uninformative. It is evaluated on exactly the same target cases as the signal-dependent methods.

Using this control does not imply that a fixed center is a usable tracking system. It instead establishes a minimum diagnostic standard for the mean-distance metric: a complex method should demonstrate value beyond the bounded geometry alone before an application claim is made. Other downstream objectives, such as multiple-person detection, remain outside this experiment.

VIII. Experimental Design and Metrics

A. Calibration recovery experiment

The generating distribution uses $p = .548$, $\sigma_1 = .971$, and $\sigma_2 = 3.003$ dB. Five calibration sizes are evaluated: 100, 500, 1,000, 5,000, and 10,000. Each size has 20 independent replicates. For each parameter θ_k, recovery is summarized by

$$\mathrm{MAE}(\widehat{\theta}_k) = \frac{1}{20} \sum_{b=1}^{20} |\widehat{\theta}_{k,b} - \theta_{k,0}|. \tag{26}$$

The convergence rate is the fraction meeting the parameter-change stopping rule before the limit. A larger sample can reduce estimation error without uniformly reducing the number of EM iterations. These are different outcomes and are reported separately.

This experiment evaluates recovery of a known, correctly specified density. It does not compare EM with another parameter optimizer, estimate the model's real-data goodness of fit, or demonstrate that a two-component model is preferable to a one-component model in a new environment. Such claims would require their own objective, competing fits, and held-out measurements.

B. Initialization experiment

A separate fixed calibration sample is fitted from 20 random starts. For each start, the stored diagnostics contain initial values, final estimates, iterations, convergence flag, likelihood per sample, and floor hits. Since the dataset stays fixed, variation in these fits concerns initialization rather than variation from collecting another dataset.

Ordering the components before comparing final values avoids an artificial label-switching difference. Agreement of final values and likelihoods is evidence that the chosen starts reach nearly the same solution for this sample. It is not a proof that all initializations or all possible datasets have the same behavior. Boundary starts, nearly identical component spreads, and very short calibration sequences are not exhaustively explored.

C. Localization experiment

The main run uses seed 7, 5,000 calibration residuals, and 500 target pixels sampled uniformly with replacement. Every target has an independent noisy query. All signal-dependent methods receive the same query in a case, and the center control is scored against the same target. Sampling with replacement allows repeated pixels; the 500 cases are not 500 distinct locations or physical acquisition sessions.

For case j with true coordinate q_j and predicted coordinate $\widehat{q}_j$, the error is $e_j = \|\widehat{q}_j - q_j\|_2$. The reported metrics are

$$\bar{e} = \frac{1}{T} \sum_{j=1}^{T} e_j, \tag{27}$$

$$\mathrm{RMSE}_{\mathrm{position}} = \sqrt{\frac{1}{T} \sum_{j=1}^{T} e_j^2}. \tag{28}$$

Both are measured in meters. RMSE emphasizes larger errors more strongly. For the two image reconstructors the output also records mean pixel-wise squared image error in dB^2. A coordinate-only predictor has no corresponding reconstructed image, so its map error is left undefined rather than set to zero.

D. Bootstrap uncertainty and paired differences

Mean-error intervals are calculated by resampling case indices with replacement and taking percentile bounds from bootstrap means. For a comparison of methods A and B, each bootstrap sample reuses the same indices for both methods. The case difference is $d_j = e_{A,j} - e_{B,j}$, so a positive mean difference favors B.

Resampling methods independently would discard their pairing and answer a less precise question about the same generated cases. However, pairing does not remove model bias. The resulting intervals remain conditional on the fixed geometry, calibration fit, generating model, and reference database. A claim about future buildings would require replication across buildings or a defensible hierarchical sampling model.

The center control was added during audit because it materially changes interpretation. This is transparent retrospective analysis, not a prospectively registered success test. The same cases and earlier metric values are preserved. Before a confirmatory follow-up, the outcomes, development/test split, comparison methods, and decision criteria should be fixed in advance.

E. Provenance of the local run

All numerical values in the Results derive from the current `results/python_preliminary` directory: `parametros.json`, `metricas.csv`, `em_sample_size.csv`, `em_initialization.csv`, `predicciones.csv`, and `comparaciones_pareadas.csv`. The report does not merge separate multipath pilots or historical output folders with this evaluation.

The main parameter file records the selected C++ EM backend and its compiler/source information alongside Python, NumPy, Matplotlib, and the Windows execution environment used by the auxiliary driver. The recorded compiler is `g++` 16.1.0, with C++17, optimization, warning, and static-linking flags retained in the backend manifest. These records describe the executed pipeline, not a claim that the whole RTI application has been ported to C++. The seed and configuration support rerunning the synthetic computation, while saved case-level predictions allow the metrics to be checked without regenerating the random stream. Software versions do not imply identical wall-clock measurements elsewhere.

IX. Results

A. Parameter recovery as sample size increases

Table III and Fig. 2 summarize the 20 replicates at each size. The MAE of p decreases from .1048 at $N = 100$ to .01095 at $N = 10{,}000$. The corresponding standard-deviation MAEs decrease from .1671 to .01489 dB for the narrower component and from .4036 to .03252 dB for the wider component. This is the strongest direct result about EM in the current study: under the generating model, larger calibration samples generally recover its parameters more accurately.

TABLE III
EM recovery over 20 independent replicates per size

N	Conv. rate	MAE p	MAE σ_1 (dB)	MAE σ_2 (dB)
100	.95	.10478	.16715	.40361
500	1.00	.03751	.05091	.12299
1,000	1.00	.03811	.05010	.11400
5,000	1.00	.01575	.01944	.04785
10,000	1.00	.01095	.01489	.03252

The sequence is not strictly monotonic for every parameter. For example, the p MAE at $N = 1{,}000$ is slightly larger than at $N = 500$. With only 20 repetitions per size, such fluctuation should not be described as an algorithmic reversal. A smooth trend through the five points is a visual summary, not a measured convergence-rate law in N.

At $N = 100$, 19 of 20 fits meet the stopping rule. All fits do so at the four larger sizes. This convergence flag concerns parameter stability, not exact agreement with the generating parameters. The average iteration counts are 182.25, 147.6, 159.5, 144.6, and 141.9, respectively. Increasing sample size therefore reduces recovery error much more clearly than it orders computational iteration counts.

B. Main fit and initialization sensitivity

The 5,000-residual main fit gives $\widehat{p} = .5618915$, $\widehat{\sigma}_1 = .9704574$, and $\widehat{\sigma}_2 = 2.9933068$ dB. It satisfies the stopping rule after 145 iterations. Its numerical Fisher information is .4182318686 dB^{-2}. These values are estimates for one generated calibration sample and need not equal the parameters used to generate the sample exactly.

The initialization diagnostic uses another fixed sample. Across 20 starts, the fitted values are nearly identical: approximately $p = .5347$, $\sigma_1 = .9163$, and $\sigma_2 = 2.8911$ dB, as shown in Fig. 3. Likelihoods per sample also agree within numerical rounding. The difference from the main fit follows from using a different calibration sample; it is not a contradiction or a repeated fit of the same residuals.

Agreement across these starts supports reproducibility with respect to the tested initializations. It does not justify calling the estimate globally optimal. The source convergence theory requires conditions on the objective and parameter sequence; trying 20 points cannot establish those conditions for every possible sample [3]. The diagnostic is evidence of practical behavior within its stated range.

C. Fisher integral and source discrepancy

Direct numerical evaluation of Eq. (23) using the published rounded values $(.548, .971, 3.003)$ gives approximately .4044137 dB^{-2}. The source text reports .548 and its example-bound table reports .5483 [1]. This discrepancy between the

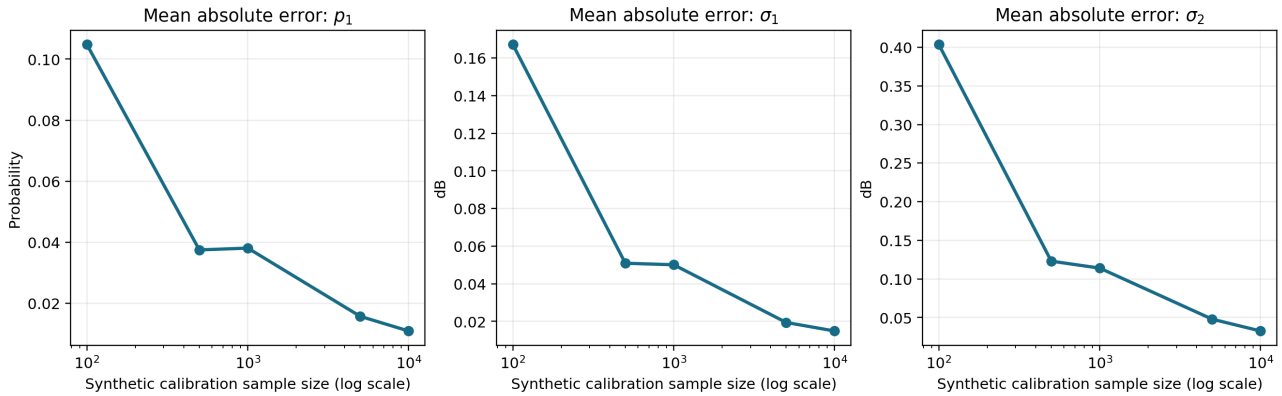


Fig. 2. Parameter-recovery MAE from 20 synthetic fits at each calibration size. Each parameter is scored against its generating value. Curves summarize a finite experiment; they do not establish a universal sample-complexity bound or performance on measured RSS. Source: `em_sample_size.csv`.

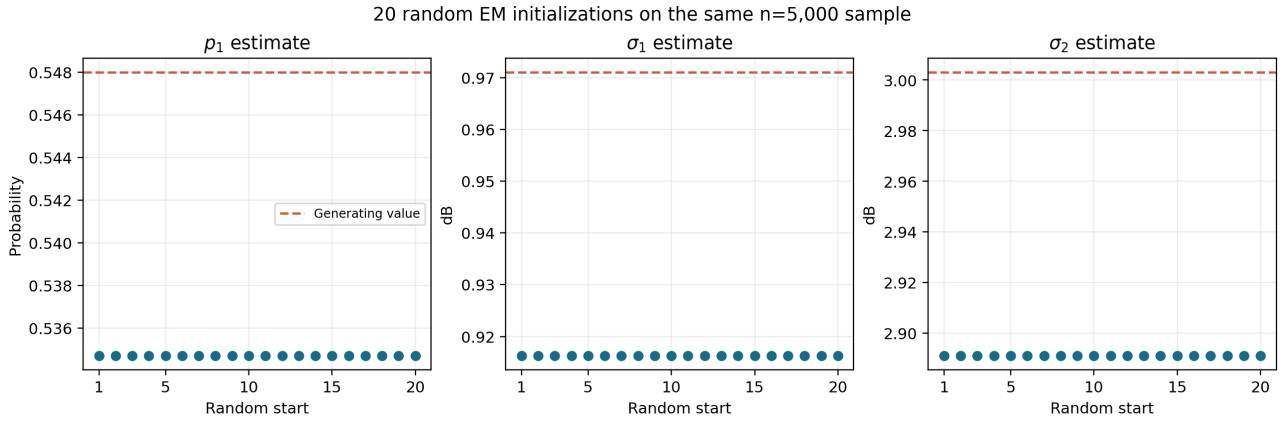


Fig. 3. Final estimates from 20 starts on one fixed synthetic calibration sample. Dashed lines denote generating parameters. Their difference from the nearly coincident fitted points reflects finite-sample estimation; agreement across starts is not a global-optimum certificate. Source: `em_initialization.csv`.

printed parameters, integral, and reported scalar remains unresolved in this project. No cause is assigned without supporting information from the authors or source implementation.

The main fitted synthetic parameters are different, and their integral gives .4182319. Its Gaussian reference, defined as the reciprocal variance of that fitted mixture, is .2244879 dB^{-2}. A mixture's location Fisher information is not generally equal to its reciprocal variance. The reference is therefore a different quantity, not another numerical estimate of the same integral.

The script integrates on a symmetric grid with 40,001 points from $-10\widehat{\sigma}_2$ to $10\widehat{\sigma}_2$, with a density safeguard in the denominator. The integral defines a property of the fitted density. It is not independently learned as a fourth free mixture parameter. Its sampling uncertainty derives from the fitted weight and spreads; the current localization run does not propagate that uncertainty by repeatedly recollecting calibration data.

D. Localization and the constant control

Table IV and Fig. 5 report all four predictors on the same 500 cases. The constant-center control has mean error 2.406646 m and position RMSE 2.562239 m. It outperforms every signal-dependent predictor in both of these aggregate metrics. Weighted K-NN is the best of the signal-dependent methods, but its mean error of 2.618212 m remains larger than the center control's. Figure 4 separately illustrates one generated image and its three signal-dependent outputs; it is not the aggregate error comparison.

TABLE IV
POSITION ERRORS ON 500 PAIRED SYNTHETIC CASES

Predictor	Mean (m)	RMSE (m)
Classical Tikhonov RTI	3.244120	3.840000
Scalar Fisher extension	3.061814	3.697517
Weighted K-NN	2.618212	2.890830
Constant room center	**2.406646**	**2.562239**

The Fisher extension improves the classical Tikhonov mean by .182306 m, about 5.62%. That reduction does not establish improved exploitation of mixture structure because Eq. (25) identifies it with a change in regularization. It also does not overcome the center-control deficit. A positive internal RTI comparison and a poor overall localization benchmark can both be true in the same evaluation.

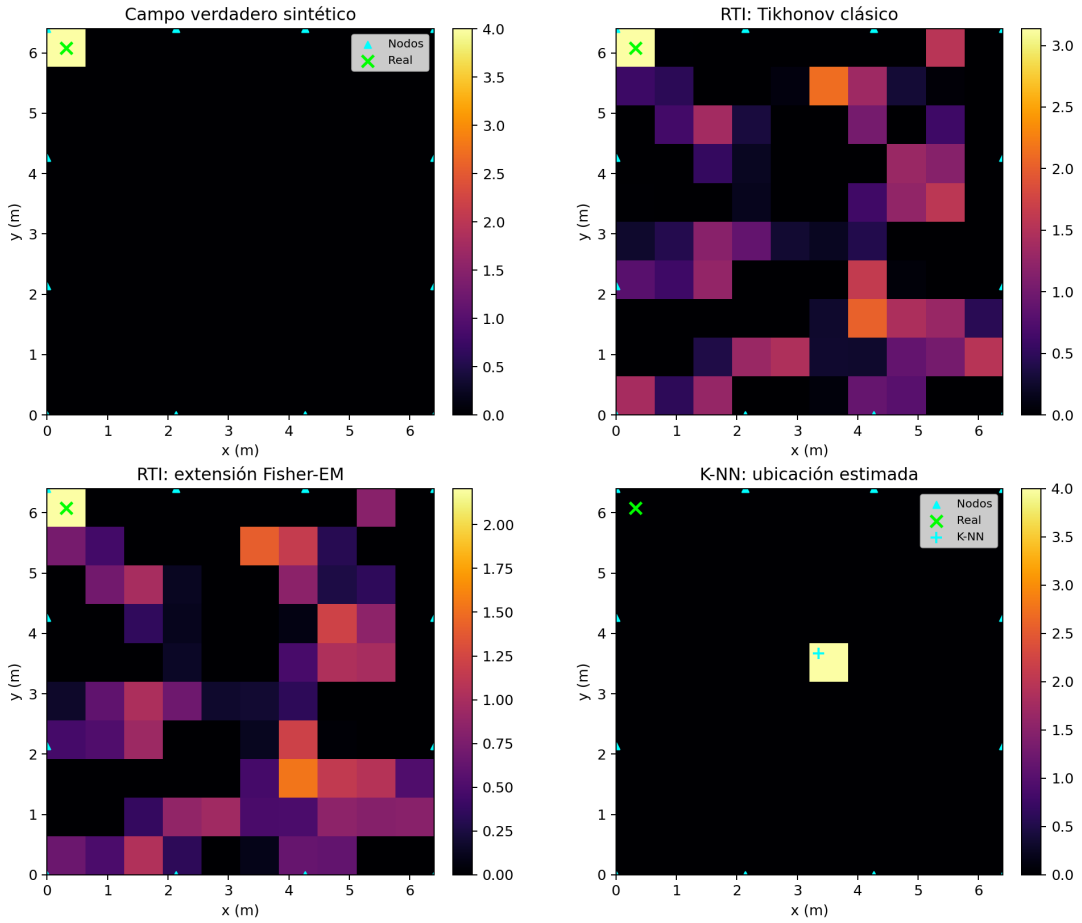


Fig. 4. One illustrative generated case: true single-pixel image (top left), classical Tikhonov reconstruction (top right), scalar Fisher extension (bottom left), and weighted K-NN position (bottom right). The supplied source plot retains its Spanish labels. Color scales differ between panels, and the K-NN panel displays a coordinate prediction rather than a reconstructed attenuation field. This example is not an aggregate accuracy result. Source: the current preliminary-results comparison PNG.

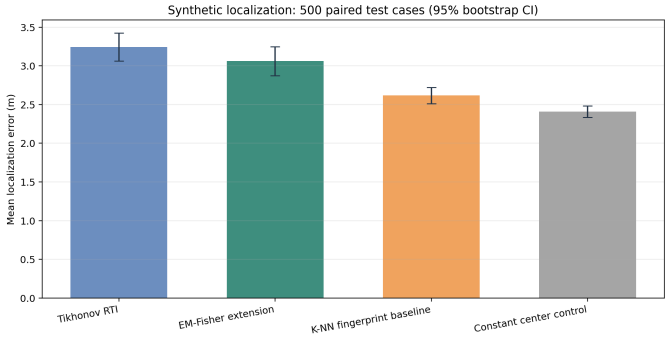


Fig. 5. Mean position error on 500 paired synthetic cases; error bars are 95% case-bootstrap intervals. The constant room-center control is included. Lower is better. Source: the current `metricas.csv` and its generated aggregate comparison plot.

The classical RTI image MSE is .801107 dB^2, compared with .509575 dB^2 for the extension. These numbers concern map amplitudes averaged over pixels. Position error instead uses the maximum pixel. Smoothing can reduce an image-error metric while leaving a broad or incorrectly placed maximum. A map metric and a localization metric should therefore be reported together rather than substituted for one another.

The 95% case-bootstrap interval for classical RTI mean error is [3.0642, 3.4228] m; for the extension it is [2.8723, 3.2460] m; and for weighted K-NN it is [2.5098, 2.7223] m. These intervals quantify the stored generated cases under the conditioning described above. They should not be presented as uncertainty bounds for a deployed system.

E. Paired comparisons

The paired mean reduction from classical RTI to the extension is .182306 m, with a bootstrap interval [.067279, .299317] m. The reduction from classical RTI to weighted K-NN is .625908 m, with interval [.420143, .832176] m. From the extension to weighted K-NN it is .443602 m, with interval [.235722, .649831] m. These positive intervals favor the stated candidates relative to their paired baselines on the generated cases.

They do not imply that the extension wins against a tuned Tikhonov family, that EM wins against K-NN, or that either

method beats the no-signal control. The control is deliberately displayed in the main metric table and figure so that a favorable subset of pairwise comparisons cannot replace the full benchmark interpretation. The saved case-level predictions are the appropriate input for any additional paired comparison.

X. Interpretation and Threats to Validity

A. What the EM result means

The recovery experiment provides evidence that the implementation performs its intended restricted estimation task. It computes soft component assignments and updates a mixing probability and two second moments. Across the tested calibration sizes, its error generally decreases with more generated observations. This result establishes calibration behavior under the stated assumptions; it does not establish an advantage over a localization rule solving a different task.

Parameter estimation and target localization have different ground truths. For calibration, the known parameters define an unambiguous recovery score. For localization, a fitted density is only one part of a pipeline that also depends on geometry, target representation, regularization, and the coordinate extraction rule. A well-fitted noise model cannot by itself correct a poor spatial forward model or a strongly biased position estimator.

B. Why a scalar is insufficient for robustness

The current γ summarizes a density globally. It does not vary with the observed residual of a particular query link. Equation (25) consequently places the extension inside the Tikhonov regularization family. The observed mean reduction is consistent with choosing a different smoothing strength. Attribution of this reduction specifically to latent mixture separation would require a comparison that controls for that strength.

A different future estimator could use the query residuals $y - Wx$ to compute posterior link responsibilities and update link-specific precisions while estimating the image. That would be another model and another iterative inverse problem. It is a proposed follow-up, not the estimator evaluated here. Its behavior under contamination would need to be measured, with clearly defined contamination and held-out configurations, before making a robustness claim.

Likewise, increasing the wide-component variance is only a statistical stress condition. It does not simulate a wall of a particular density, a moving reflector, a blocked link's packet loss, or coherent multipath. The appropriate language is "generated mixture noise" unless a physical propagation or measured-channel model supplies a more specific interpretation.

C. Implication of the center-control result

A bounded room offers prior positional information even without signals. Always predicting its center can have a moderate average error because every target is within a limited region. In this run, the signal-dependent methods do not convert observations into a lower mean distance error than that spatial prior. Their predictions may still differ case by case, but the principal reported utility metric does not favor them.

The deficit motivates inspecting case distributions and model settings before expanding to rescue scenarios. Potential contributors include the single-pixel target, sparse geometric sensitivity, noise amplitude relative to signal, smoothing choices, and the maximum-pixel rule. The current evidence does not isolate their relative contributions. Naming them as hypotheses is different from claiming to have diagnosed a measured radio failure.

Adding the control identifies a negative application result that a comparison among signal-dependent predictors alone would conceal. The generated localization benchmark does not demonstrate utility beyond the spatial prior. Independent development/test evaluation and repeated configurations are required before changing that conclusion. Selecting only seeds or noise regimes favoring one method would answer a different, selectively defined question.

D. Comparison fairness and data leakage

K-NN receives labeled references at every grid cell, including cells that can recur as test targets. Independent noise draws prevent identical training vectors, but the evaluation is spatially matched to the reference map. This is a legitimate matched-grid fingerprint test if described as such. It is not a held-out-position generalization test and should not be labeled one.

RTI uses a hand-selected ellipse width and smoothing coefficient, while K-NN uses a fixed neighborhood and three references per cell. None of these hyperparameters is selected through a separate development set in the current comparison. A fairer future study would allocate a development set, tune each method's parameters using only that set, and retain a test set untouched until the settings are fixed.

Equal test observations do not remove unequal setup requirements. A fingerprint database requires labeled positions; an RTI matrix requires surveyed geometry and a model. A future utility comparison should measure those acquisition and maintenance costs. The present synthetic generator makes both inexpensive to construct, so it cannot answer which approach would require less field work.

E. Statistical and physical limitations

The experiment uses one room geometry, one main calibration fit, one generating noise family, and one fixed main seed. Its 500 cases vary target selection and query noise within that configuration. Treating them as 500 independent building deployments would overstate the evidence. Replicating the entire calibration–training–query process across seeds would address a broader computational population, while real acquisition would address another level of external validity.

Noise is independent across links and draws. Actual RSS time series can be correlated, link-specific, quantized, censored by packet reception, and affected by transmitter power, antenna orientation, channel occupancy, and offsets. These effects can alter both the calibration distribution and the observation model. They are omitted rather than estimated implicitly.

The source's published γ discrepancy also limits a claim of exact reproduction. This report reproduces the printed model formulas as interpreted in code and documents the numerical inconsistency. It does not resolve it by inventing precision, changing reported values, or equating the source's theoretical bound with the implemented image estimator.

F. Timing limitations

The local outputs contain exploratory timings. They are not used here to claim one method is faster. Dense RTI has precomputation, K-NN has reference construction, and EM calibration occurs before querying. The C++ EM diagnostic timers include process startup and serialization through the adapter; compilation is performed beforehand. Those times therefore do not isolate the cost of the C++ core. A meaningful comparison would define setup and inference consistently, warm the numerical libraries, repeat measurements, and report distributions under a controlled workload.

In particular, comparing precomputed RTI multiplication against an unprepared nearest-neighbor search measures a pipeline choice as well as an algorithm. The asymptotic analysis of the EM core remains valid as an operation-count statement. It should not be converted into a hardware-specific latency claim from a single run.

XI. A Focused Follow-up Protocol

A. Development and test separation

The immediate computational follow-up should retain the current four methods and add the algebraically matched Tikhonov control. First define independent calibration, development, and test random streams. Use calibration only for noise fitting. Use development cases to choose α, ellipse width, and K-NN settings. Freeze those choices before generating or opening test summaries.

Two spatial tasks should be distinguished. A matched-grid task allows test locations to have reference fingerprints, with independent noise draws. A held-out-position task excludes test positions from the fingerprint map and states how interpolation is assessed. Reporting them separately prevents a method's success under one task from being interpreted as performance under the other.

The first decision criterion is mean-distance improvement over the center control, accompanied by RMSE and a distribution of errors. The second is whether an EM-based method improves over an independently tuned Tikhonov method. The third is whether any improvement persists across full-pipeline repetitions, rather than a single favorable calibration fit. No numerical threshold is declared achieved by the present study.

B. Calibration-focused follow-up

For an EM-focused calibration study, a follow-up can vary separation of the two spreads, narrow-component probability, sample size, initialization, or stopping tolerance while keeping the task identifiable. Each factor requires a stated hypothesis and a corresponding summary of recovered parameters and convergence behavior.

For example, bringing the two spreads close together reduces the practical distinction between components. It may become harder to recover the individual parameters even when the fitted density predicts held-out residuals reasonably well. Measuring both parameter error and predictive log likelihood would reveal that difference. This proposed experiment would teach a more precise limitation of mixture estimation than simply labeling a localization result as an EM success or failure.

Another useful comparison would fit a single zero-mean normal and the two-component mixture, then compare held-out residual likelihood. That comparison tests whether the added mixture flexibility improves prediction under specified conditions. It must use independent data because a more flexible model can improve training likelihood without improving future predictions. These are planned analyses, not reported results.

C. If physical acquisition becomes available

The first measurement session should use an open, bounded area with surveyed node positions and a stable set of links. Log timestamps, sender and receiver identifiers, RSS, channel, transmit power, packet success, and calibration state. Record an empty-area baseline before introducing a person at known positions. This establishes what the available hardware can actually measure before adding walls or obstacles.

Link-wise offset removal and exploratory distributions would then precede mixture fitting. Repeated sessions would allow a distinction between within-session noise and changes across sessions. Test positions and sessions should be held out according to the intended use. A physical experiment may show that another density, target model, or geometry is needed; preserving that possibility is part of evaluating the method.

Only after those checks would obstacle experiments be interpretable. An eventual search-and-rescue motivation requires substantially more evidence about propagation, availability of a baseline, deployment access, motion, false alarms, and field conditions. None of those requirements is resolved by the current synthetic parameter-recovery result.

XII. Reproducibility

A. Editable source and compilation

The manuscript uses the official IEEEtran conference class and continuous two-column LaTeX flow. The editable package contains `main.tex`, `refs.bib`, `enlaces.tex`, figures, source-code listings, and the unmodified class and bibliography style. Its main document is `main.tex`; the bibliography uses BibTeX with the IEEEtran style.

The PDF is a project manuscript. Its use of the IEEEtran layout does not imply publication or peer review. The source package and provenance manifest permit independent inspection of the document, figures, implementation, and input-result files.

B. Reproduction sequence

The EM source is `em.cpp`, compiled as C++17. The auxiliary experiment command is `python simulacion_rti_em_knn.py`, using the documented C++ backend defaults. The adapter executes the compiled EM routine for calibration and its repeated diagnostics. Python writes the local result tables, carries out the surrounding RTI computations, and produces comparison plots. A Python reference fit is available for implementation validation and is identified explicitly when selected.

The explicit build command is `python compilar_cpp.py`; the helper invokes `g++` and records its actual flags and version. Running `build/em.exe -demo` executes the four-residual, one-iteration example in C++. The source requires only the C++ standard library. Its command-line fitting mode accepts residuals and initial settings, so the calibration algorithm can also be exercised independently of the RTI driver.

Rerunning the experiment overwrites generated outputs in the selected output directory, so an archived result snapshot should be kept when comparing manuscript versions. Reported numerical claims should always identify a single coherent run. The manuscript package preserves copies of its plotted figures, and its provenance note identifies their source files rather than synthesizing historical outcomes from different runs.

An independent reader can check the error table directly from saved predictions, the EM diagnostic table from its replicate summary, the mixture density from the fitted parameters, and the scalar equivalence by factoring γ from the normal equations. These checks cover different failure modes and are more informative than treating the existence of a PDF as evidence that the computation is correct.

C. Implementation and artifact availability

The EM implementation is C++17. Auxiliary Python scripts support the synthetic application and numerical validation. The repository, manuscript workspace, and recorded-presentation links are listed in Appendix D; a pending entry indicates that a public URL is not yet available.

XIII. Conclusion

The restricted C++17 EM implementation recovers known mixture parameters with decreasing error as calibration sample size generally increases, and the tested random starts yield nearly identical estimates on one fixed sample. These are direct, reproducible findings about the algorithm under its own generating assumptions. The derivation and numerical example explain why responsibilities and weighted second moments produce those updates. Python utilities support the surrounding synthetic application and the reference cross-checks.

The localization result is substantially weaker. Mean errors are 3.244 m for classical Tikhonov RTI, 3.062 m for its scalar Fisher extension, 2.618 m for weighted K-NN, and 2.407 m for the constant room-center control. Every signal-dependent method loses to that control in mean error. Moreover, the scalar extension is exactly a Tikhonov regularization change after rescaling, so its improvement cannot be attributed to link-specific robustness or a new EM localization mechanism.

The contribution is a transparent study of EM noise calibration connected to a limited synthetic RTI application. Useful localization, real Wi-Fi sensing, NLoS robustness, missing-data reconstruction, and obstacle penetration remain unestablished. Independent parameter tuning and a control-aware follow-up should precede broader application claims.

Appendix A
The C++17 EM Core

Listing 1 contains the EM core extracted directly from `em.cpp`. It uses log-domain normalization, spread flooring, label ordering, and a relative-parameter stopping rule. The complete editable C++17 source is included as `codigo/em.cpp` in the manuscript package. It also contains the standard-library includes, result structure, command-line interface, and demonstration surrounding the core.

The four-sample worked example can be checked through the C++ demonstration using $(-3, -1, 1, 3)$ and exactly one iteration initialized at $(.5, 1, 3)$. A complete fit should use an adequate sample and the configured stopping rule. A four-observation illustration is not evidence of reliable calibration. The compiler and source hash are recorded alongside the generated results and in the report provenance manifest.

Appendix B
Equation-to-Code Mapping

The input vector contains observed residuals, and the three scalar state variables are the parameters being estimated. The code does not receive latent labels from the generator. In `responsibility`, `a` and `b` represent log component probabilities, `largest` stabilizes exponentiation, and `ea` and `eb` provide normalized exponential terms. The E-step stores the returned narrow-component probabilities in the `tau` vector.

In the M-step, `mass1` and `mass2` are fractional sample counts. An explicit loop accumulates `square_sum1` and `square_sum2`, the weighted squared residuals. Dividing by the masses gives component variances because the means are fixed at zero. The square roots restore the standard-deviation parameterization. The `change` calculation measures whether the three parameters have stabilized relative to their previous magnitudes.

The floor and probability clipping restrict the parameter domain. The component swap changes names rather than the density, provided the probability is complemented at the same time. The convergence flag indicates whether the requested stopping condition was satisfied; it does not replace examination of the fitted density, sample adequacy, or the likelihood surface.

This mapping separates the latent-label estimation problem from the RTI geometry. The EM function receives residuals and returns three fitted noise parameters and convergence information. Image reconstruction and coordinate prediction are downstream computations with different inputs, outputs, and computational costs.

Listing 1. Executed C++17 EM core for two Gaussian components with means fixed at zero. Extracted from the packaged editable source.

```cpp
// Paso E: probabilidad posterior de la componente estrecha, en logaritmos.
double responsibility(double x, double p, double s1, double s2) {
    const double v1 = std::max(s1 * s1, 1e-8), v2 = std::max(s2 * s2, 1e-8);
    const double pi = std::acos(-1.0);
    const double a = std::log(std::max(p, 1e-12)) - 0.5 * std::log(2 * pi * v1) - x * x / (2 * v1);
    const double b = std::log(std::max(1 - p, 1e-12)) - 0.5 * std::log(2 * pi * v2) - x * x / (2 * v2);
    const double largest = std::max(a, b);
    const double ea = std::exp(a - largest), eb = std::exp(b - largest);
    return ea / std::max(ea + eb, 1e-300);
}

// Entradas: residuos finitos, limite/tolerancia, inicios y piso de sigma.
// Un inicio sigma NaN solicita el valor automatico basado en std poblacional.
// Salida: p, sigma1, sigma2, iteraciones y cumplimiento de la tolerancia.
FitResult fit_em(const std::vector<double>& x, int max_iterations, double tolerance,
                 double p, double s1, double s2, double sigma_floor) {
    const double n = static_cast<double>(x.size());
    if (x.empty()) throw std::invalid_argument("Se requiere al menos un residuo.");
    const double mean = std::accumulate(x.begin(), x.end(), 0.0) / n;
    double sum_deviations = 0;
    for (double value : x) sum_deviations += (value - mean) * (value - mean);
    const double spread = std::max(std::sqrt(sum_deviations / n), 1e-3);
    p = std::clamp(p, 1e-4, 1.0 - 1e-4);
    s1 = std::max(std::isnan(s1) ? spread * 0.55 : s1, sigma_floor);
    s2 = std::max(std::isnan(s2) ? spread * 1.8 : s2, s1 * 1.05);
    std::vector<double> tau(x.size());  // O(N) memoria de responsabilidades.

    for (int iteration = 1; iteration <= max_iterations; ++iteration) {
        const double old_p = p, old_s1 = s1, old_s2 = s2;
        // E: cada residuo recibe una pertenencia suave entre cero y uno.
        for (std::size_t i = 0; i < x.size(); ++i)
            tau[i] = responsibility(x[i], p, s1, s2);
        const double mass1 = std::max(std::accumulate(tau.begin(), tau.end(), 0.0), 1e-8);
        const double mass2 = std::max(n - mass1, 1e-8);
        // M: actualiza peso y segundos momentos ponderados; medias fijas cero.
        p = std::clamp(mass1 / n, 1e-4, 1.0 - 1e-4);
        double square_sum1 = 0, square_sum2 = 0;
        for (std::size_t i = 0; i < x.size(); ++i) {
            const double square = x[i] * x[i];
            square_sum1 += tau[i] * square;
            square_sum2 += (1 - tau[i]) * square;
        }
        s1 = std::max(std::sqrt(std::max(square_sum1 / mass1, 0.0)), sigma_floor);
        s2 = std::max(std::sqrt(std::max(square_sum2 / mass2, 0.0)), sigma_floor);
        if (s1 > s2) { std::swap(s1, s2); p = 1 - p; }
        const double change = std::max({
            std::abs(p - old_p) / std::max(std::abs(old_p), 1e-8),
            std::abs(s1 - old_s1) / std::max(std::abs(old_s1), 1e-8),
            std::abs(s2 - old_s2) / std::max(std::abs(old_s2), 1e-8)});
        if (change < tolerance) return {p, s1, s2, iteration, true};
    }
    return {p, s1, s2, max_iterations, false};
}
```

APPENDIX C
ARTIFACT AND CLAIM TRACEABILITY

Table V maps the reported results to the current experiment snapshot. Paths are repository-relative; `provenance.json` records input, source, and figure hashes.

Figure 4 uses the current preliminary-results comparison PNG; the other plots correspond to the diagnostic and metric files identified in their captions.

APPENDIX D
DELIVERY LINKS

Project artifact links are listed below. Pending entries indicate unavailable URLs.

Overleaf manuscript: *Pending overleaf URL*

GitHub repository: https://github.com/Camimilo-11/Wireless-Save

TABLE V
TRACEABILITY OF NUMERICAL AND METHODOLOGICAL CLAIMS

Claim or artifact	Evidence origin
Generating mixture values	Source table 1 [1]; recorded parameter JSON
Main fitted parameters	`parametros.json`
Sample-size recovery	`em_sample_size.csv`
Initialization agreement	`em_initialization.csv`
Position mean and RMSE	`metricas.csv`; case predictions
Paired error reductions	`comparaciones_pareadas.csv`
Figures and source hashes	`provenance.json`
α/γ equivalence	Algebra of Eqs. (24)–(25)

Recorded presentation: *Pending video URL*

REFERENCES

[1] J. Wilson and N. Patwari, "Radio tomographic imaging with wireless networks," *IEEE Transactions on Mobile Computing*, vol. 9, no. 5, pp. 621–632, 2010.

[2] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–22, 1977.

[3] C. F. J. Wu, "On the convergence properties of the EM algorithm," *The Annals of Statistics*, vol. 11, no. 1, pp. 95–103, 1983.

[4] P. Bahl and V. N. Padmanabhan, "RADAR: An in-building RF-based user location and tracking system," in *Proceedings IEEE INFOCOM*, vol. 2, 2000, pp. 775–784.